slot-mate — arhitectura țintă (ce trebuie să fie la final, după U5 și F17)

Doar Docker: docker compose cu profiluri, noțiunile din car-platform traduse fără Kubernetes · F = feature-ul de backend, U = pasul de interfață din PLAN.pdf · eticheta = profilul compose · chenar punctat = doar local / la demo

Browser (Internet)

DNS (F6)
api · app · auth · grafana · kibana · kafka-ui .<domeniu>
A record → Elastic IP

EC2 Elastic IP
SG: 80, 443 deschise · 22 doar IP-ul tău

Clienți: slot-mate-client în browser (U1–U5)
Swagger UI (F6) · MCP Inspector (F9) · token de la Keycloak din F16

Model AI (F12) · Ollama, doar local / demo
chemat de asistent prin ChatClient · pe EC2: fallback pe reguli

GitHub · monorepo-ul slot-mate (F1b)
Actions pe folder: api · agent · notifier · ui · ops
test → image → deploy (ssh)

GHCR · ghcr.io/tomescuflavius/slot-mate-* :SHA

Docker Engine · docker compose (slot-mate-ops/docker-compose.yml) · pe EC2: core · edge · obs · auth · local și la demo: toate profilurile

edge · rețeaua Docker *edge* · singurele porturi publicate: 80 / 443 (nginx)

nginx + certbot (F6) edge · TLS Let's Encrypt
api · app · auth · grafana · kibana · kafka-ui
api.* + app.*/api → Kong · app.* / → ui
doar TLS + reverse proxy, fără logică

ui · slot-mate-client (U1–U5) core
React + Vite, build static servit de nginx

Kong — API gateway (F17) edge
DB-less: kong.yml în Git (kong-init)
/api · /mcp → api · /api/assistant → agent
/api/notifications → notifier
jwt (Keycloak) · rate-limiting · correlation-id · prometheus

oauth2-proxy (F16) auth
auth_request din nginx
grafana · kibana · kafka-ui
doar grupul /admins

grafana · kibana · kafka-ui

auth (F16) · Keycloak = sursa conturilor și a rolurilor

Keycloak auth
start --import-realm · KC_HOSTNAME fix

Postgres keycloak-db auth
volum + healthcheck

realm-slot-mate.json în Git · 13 permisiuni = roluri · CLIENT / PROVIDER / ADMIN / AGENT compuse → claim permissions · clienți: slot-mate-app · slot-mate-ui (PKCE, U5) · slot-mate-api · slot-mate-agent · oauth2-proxy

OIDC

MCP prin Kong: citire + proposeSlotPlan

aplicațiile tale · rețeaua Docker *backend*
fără porturi publicate · intră doar prin Kong (F17)

slot-mate-api · monolitul (F1–F10, F13) core

REST /api (F1–F8, F10)

Server MCP · Spring AI (F9–F10)

Spring Security · JWT HS256 (F2) → **resource server pe Keycloak** (F16) · hasAuthority = matricea

category (F1) · users (F2, F16) · provider (F3) · offering (F4) · **slot** (F5) publishSlot
booking (F6) · program + job (F7) · ocupare (F8) · propuneri (F10)

Producer **booking.created · booking.cancelled** (F13), după commit · EVENTS_ENABLED=false pe EC2

Logback JSON (ecs) + MDC runId / requestId = X-Correlation-ID (F7, F17) · Actuator + Micrometer pe 8081 (F8) · OpenTelemetry OTLP → colector → Tempo (F15)

o propunere aprobată cheamă publishSlot / deleteSlot: aceeași cale și aceeași validare ca acțiunea manuală a furnizorului; agentul doar propune

slot-mate-agent · primul microserviciu (F11–F12) core

Client MCP · cont AGENT (F11) → client credentials (F16) · prin Kong (F17)

Planificator (F11)
RuleBasedPlanner · R1 / R2 · 02:00

Asistent (F12)
ChatClient → Model AI
fallback RuleBasedSuggester

fără bază de date · POST /api/assistant/suggestions (ASSISTANT_USE) · tokenul clientului merge mai departe la /mcp · runId în MDC

slot-mate-notifier (F14) messaging

consumă booking.* · emailuri + reminder 24h (ReminderJob, 5 min)
GET /api/notifications/me · notifier_db (Flyway-ul lui)

nu știe de users: destinatarii vin din eveniment · email trimis o singură dată

messaging (F13–F14) · doar local / demo

Kafka apache/kafka · KRaft, un nod · volum messaging

topic **booking.created** · api → notifier

topic **booking.cancelled** · api → notifier

kafka-init (creează topicurile) · **Kafka UI** (kafbat, prin oauth2-proxy) messaging

Mailpit · SMTP de test, UI 8025 doar local messaging

contractul dintre servicii = topicurile (events.yaml, AsyncAPI) · cheia = slotId · consumatori idempotenți: UQ (event_id, recipient_email, type) în notifier_db

data · volume Docker numite

MySQL 8.4 · slot_mate_api_db core · JPA din api
Flyway V1–V11 · ddl-auto: validate · healthcheck → depends_on: service_healthy

notifier_db (F14) · Flyway-ul notifier-ului · fără FK spre API messaging

Postgres-ul Keycloak e în profilul auth · parolele doar în .env pe EC2, în Git doar .env.example

monitoring (F8 · F15)

Prometheus obs · Grafana obs
colector OTel (F15) tracing · Tempo (F15) tracing

scrape: api:8081, agent, notifier, Kong (status) · Grafana provisioning din Git · ocupare, rezervări, conflicte, 429

logging (ELK · F15) · doar local / demo

Elasticsearch logs · Kibana logs
Filebeat · citește /var/lib/docker/containers logs

logurile JSON din toate containerele → Elasticsearch → Kibana · căutare după bookingId / runId / traceId

Ce ține locul Kubernetes (ca la cars, fără cluster)

rețele Docker edge · backend
Kong, Keycloak, Grafana, Kibana în ambele; nginx, ui doar edge

profiluri compose (din F1)
core · edge · obs · auth · messaging · logs · tracing

healthcheck + depends_on service_healthy
restart: unless-stopped · limite de memorie

config în Git: kong.yml · realm-slot-mate.json · nginx/ prometheus.yml · grafana/ · filebeat.yml · .env.example

Memoria: EC2 (4 GB din F16) rulează core + edge + obs + auth; messaging, logs și tracing nu încap: rulează local și pe instanța de demo (min. 8 GB) · deploy-ul automat pornește doar COMPOSE_PROFILES din .env

Fluxul de livrare (F1b): 1. git push pe master → 2. test (Gradle + Testcontainers; ui: npm build + Playwright) → 3. image: build + push ghcr.io/tomescuflavius/slot-mate-api | agent | notifier | client :SHA → 4. deploy: scp compose, ssh pe EC2 → docker compose pull && up -d (doar profilurile din .env)

Legendă
portocaliu = edge / gateway (DNS, EC2, nginx, Kong, oauth2-proxy) · roșu = Keycloak și logurile · mov = metrici și tracing · verde = Kafka · galben = date · albastru = Docker în locul Kubernetes · violet MCS = aplicațiile tale
Partea IV: **F16** Keycloak (realm în Git, API / agent / notifier = resource server, agentul pe client credentials, register delegat, login scos, oauth2-proxy pe UI-uri) · **F17** Kong = API gateway (jwt pe Keycloak, rate-limiting, correlation-id, metrici; agentul și UI-ul trec și ele prin Kong; 401 fără ProblemDetail, Error id, 429 în UI) + rețelele Docker: aplicațiile fără porturi publicate
Partea V: ui = slot-mate-client (React + Vite + TS), fișiere statice în rețeaua edge · U1 catalog + cont (după F3) · U2 rezervări (F6) · U3 firma mea (F10) · U4 asistentul (F12) · U5 login Keycloak PKCE (imediat după F16, înainte de F17)
Ordinea: F1, F1b, F2, F3, U1, F4–F6, U2, F7–F10, U3, F11, F12, U4, F13–F16, U5, F17